

Einführung in die Diskrete Mathematik

2. Programmierübung

Implementieren Sie den Algorithmus von Borůvka, um einen kostenminimalen aufspannenden Baum in einem zusammenhängenden Graphen zu finden. Ihre Implementierung soll eine Laufzeit von $O(m \log n)$ haben, wenn m die Zahl der Kanten und n die Zahl der Knoten ist.

Das Programm muss in Python oder C++ geschrieben sein. Es wird empfohlen, C++ zu verwenden. In diesem Fall kann man zum Einlesen und Speichern der Graphen die Klasse `Graph` (eventuell modifiziert) aus dem Buch „Algorithmische Mathematik“ (Hougardy/Vygen) benutzen. Diese finden Sie (wie alle Programme aus diesem Buch) hier:

<https://www.or.uni-bonn.de/~hougardy/alma/alma2nd.html>

Außerdem dürfen Sie bei Bedarf Teile der C++-Standardbibliothek einbinden. Andere externe Bibliotheken dürfen nicht verwendet werden.

Bei Verwendung von Python dürfen Sie die Datenstrukturen, die in der Vorlesung „Algorithmische Mathematik I“ im Wintersemester 2023/2024 zur Verfügung gestellt wurden, verwenden.

Programmaufruf: Dem Programm soll beim Aufruf per Kommandozeilen-Parameter der Name einer Datei übergeben werden, in welcher der Graph gespeichert ist.

Eingabeformat: Eine gültige Datei, die einen Graphen beschreibt, hat das folgende Format:

```
Knotenanzahl
Knoten0a Knoten0b Gewicht0
Knoten1a Knoten1b Gewicht1
...
```

In der ersten Zeile steht eine einzelne natürliche Zahl, welche die Anzahl der Knoten angibt. Jede weitere Zeile spezifiziert genau eine Kante. Die ersten beiden Einträge einer Zeile sind zwei verschiedene nichtnegative ganze Zahlen, welche die Nummern der Endknoten der Kante sind. Die Reihenfolge der beiden Einträge spielt keine Rolle. Dabei nehmen wir an, dass, wenn wir n Knoten haben, die Knoten von 0 bis $n - 1$ durchnummeriert sind. Der dritte Eintrag in jeder Zeile ist eine ganze Zahl, die das Gewicht der Kante angibt. Die Kanten können in beliebiger Reihenfolge in der Datei stehen.

Sie können voraussetzen, dass der gegebene Graph einfach ist.

Ausgabeformat: Das Programm soll zuerst in einer Zeile das Gewicht des berechneten Baumes ausgeben. Dann soll in jeder Zeile eine Baumkante durch die Nummern der beiden Endknoten ausgegeben werden, wobei die kleinere Knotennummer zuerst stehen soll. Alle Baumkanten sollen aufsteigend sortiert ausgegeben werden, wobei das erste Sortierkriterium die kleinere Knotennummer und (wenn diese beiden gleich sind) als zweites Sortierkriterium die größere Knotennummer verwenden werden soll. Zum Sortieren können Sie die Sortierfunktion aus der Standardbibliothek verwenden.

Beispiel: Eine Eingabedatei für einen Graphen mit fünf Knoten kann so aussehen:

```
5
0 2 2
4 2 1
3 0 -1
4 0 4
1 2 2
3 2 6
```

Die Ausgabe des Programms muss dann so aussehen:

```
4
0 2
0 3
1 2
2 4
```

Testinstanzen finden sich auf der eCampus-Seite der Veranstaltung.

Für diese Programmieraufgabe gibt es 20 Punkte.

Abgabe: Donnerstag, 5.12.2024, 16:00 Uhr s.t. auf der eCampus-Seite der Vorlesung.